Appendix G - Changes Compared to Assignment 1

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Further Information

- Assignment 1 GitHub Release: https://github.com/chechmek/EDPO_FS26_Project/releases/tag/assignment-1
- Assignment 2 GitHub Release: https://github.com/chechmek/EDPO_FS26_Project/releases/tag/assignment-2

Overview

All Assignment 1 concepts, architecture decisions, and documentation were left unchanged after the Assignment 1 submission. No retroactive edits were made to Assignment 1 documents, BPMN models, ADRs, or service logic as such. The focus after the submission was exclusively on satisfying Assignment 2 requirements.

The only changes made to the pre-existing codebase were minimal additions to the Kafka event payloads of two services so that the new stream processors could consume them. Everything else that changed was strictly additive: new services, new processors, new scripts, and extended infrastructure configuration.

Changes to existing Services

`services/verification-service/app.py`

The `verification-notification` Kafka event payload was extended with three new fields required by the stream processors:

Field added	Purpose
`eventTime`	ISO-8601 UTC timestamp used for event-time processing and window placement
`contentId`	Stable SHA-256-derived identifier for a content URL, used as the stream-processing key
`status`	Duplicates the existing `type` field in a normalised form expected by the processors

A small helper function `_content_id_for_url()` was added to derive a deterministic `contentId` from the content URL. No existing fields were removed or renamed and no BPMN workers were changed.

`services/reporting-service/app.py`

The `report-notification` , `post-deleted` , and `objection-approved` Kafka event payloads were extended with the same set of fields:

Field added	Purpose
`eventTime`	ISO-8601 UTC timestamp for event-time processing
`contentId`	Passed through from the `postId`, used as the stream-processing join key
`status`	Normalised status string matching the processor's event type expectations

A small helper function `_utc_now_iso()` was added. No existing fields, workers, or BPMN tasks were changed.

`docker-compose.yml`

Three new services were added:

Service	Description
`sla-monitor`	Python stream processor, exposed on port `8005`
`content-ledger`	Java / Kafka Streams processor, exposed on port `8085`
`ui`	Lightweight Flask dashboard aggregating both processor APIs, exposed on port `8086`

No existing service definitions were modified.

Additions for Assignment 2

Everything listed below was newly created and did not exist at the Assignment 1 release:

Path	What it is
`processors/sla-monitor/`	Full Python SLA Monitor processor with Flask IQ API
`processors/content-ledger/`	Full Java / Kafka Streams Content Decision Ledger processor
`ui/`	Flask-based web UI proxying both processor APIs
`schemas/content-ledger/`	Avro schema definitions for `ContentEvent` and `ContentDecisionState`
`scripts/`	Demo and load-generation scripts for testing the processors
`misc/content-ledger-postman-collection.json`	Postman collection for the Content Ledger REST API
`docs/(...-assignment-2)`	Documentation reagrding Assignment 2
`README.md`	Updated to cover the full Assignment 2 stack and setup instructions